

Introduction to Computer Vision



Lec.04: Model Fitting and Optimization
2023.10.16

Today

- **Optimization**
- Numerical methods
- Robust estimation
- Graphcut

Optimization

$$\begin{array}{ll}\text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \dots, m \\ & g_i(x) = 0, \quad i = 1, \dots, p\end{array}$$

- ▶ $x \in \mathbf{R}^n$ is (vector) variable to be chosen
- ▶ f_0 is the *objective function*, to be minimized
- ▶ $f_1, \dots, f_m$ are the *inequality constraint functions*
- ▶ $g_1, \dots, g_p$ are the *equality constraint functions*

Example

某军工厂生产甲、乙、丙三种产品，生产三种产品需要A、B两种资源，其单位需求量及利润由下表1给出，问每天生产甲、乙、丙三种产品各多少，可使总利润最大？

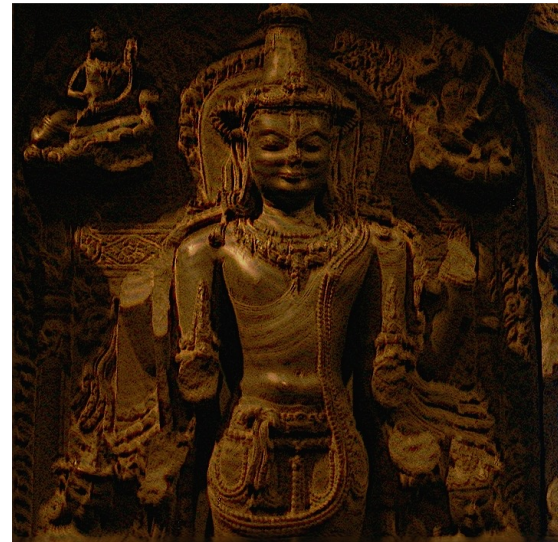
	甲	乙	丙	资源的最大量
A	2	3	1	100kg
B	3	3	2	120kg
利润	40元	45元	24元	

$$\max_{x_1, x_2, x_3} 40x_1 + 45x_2 + 24x_3$$

$$s.t. \begin{cases} 2x_1 + 3x_2 + x_3 \leq 100 \\ 3x_1 + 3x_2 + 2x_3 \leq 120 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0 \end{cases}$$

Example

- Image deblurring



$$\min_X \|Y - F * X\|^2$$

Blurred image Blur kernel Sharp image

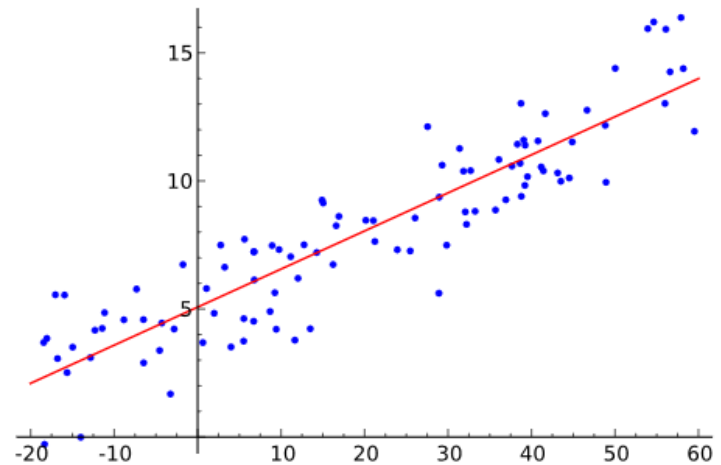
Model fitting

A mathematical model $b = f_{\mathbf{x}}(a)$

describes the relationship between input a and output b , where $\mathbf{x}$ is model parameter

Example:

- linear model $b = a^T \mathbf{x}$

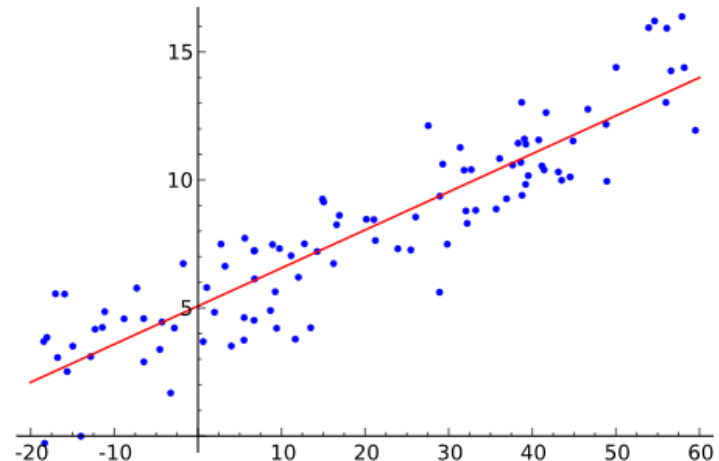


Model fitting

- How to estimate model parameters from data (usually called learning)
- A typical approach:
Minimize the **Mean Square Error (MSE)**

$$\hat{x} = \operatorname{argmin}_x \sum_i (b_i - a_i^T x)^2$$

- Why MSE?



Maximum Likelihood Estimation

- We assume the data is with Gaussian noise

$$b_i = a_i^T x + n, n \sim G(0, \sigma)$$

- Given x , the likelihood of observing (a_i, b_i)

$$P[(a_i, b_i)|x] = P[b_i - a_i^T x] \propto \exp - \frac{(b_i - a_i^T x)^2}{2\sigma^2}$$

Maximum Likelihood Estimation

- If the data points are independent

$$\begin{aligned} & P[(a_1, b_1)(a_2, b_2) \dots | x] \\ &= \prod_i P[(a_i, b_i) | x] \\ &= \prod_i P[b_i - a_i^T x] \\ &\propto \exp - \frac{\sum_i (b_i - a_i^T x)^2}{2\sigma^2} = \exp - \frac{\|Ax - b\|_2^2}{2\sigma^2} \end{aligned}$$

Maximum Likelihood Estimation

- Maximum Likelihood Estimation (MLE) =
Maximize the likelihood to find the best x

$$\begin{aligned}\hat{x} &= \arg \max_x P[(a_1, b_1)(a_2, b_2) \dots | x] \\ &= \arg \max_x \exp - \frac{\|Ax - b\|_2^2}{2\sigma^2} \\ &= \arg \min_x \|Ax - b\|_2^2\end{aligned}$$

- MSE = MLE with Gaussian noise assumption

Today

- Optimization
- **Numerical methods**
- Robust estimation
- Graphcut

Numerical methods

- Some problems have analytical solution
 - Linear MSE

$$\hat{x} = \operatorname{argmin}_x \|Ax - b\|_2^2$$

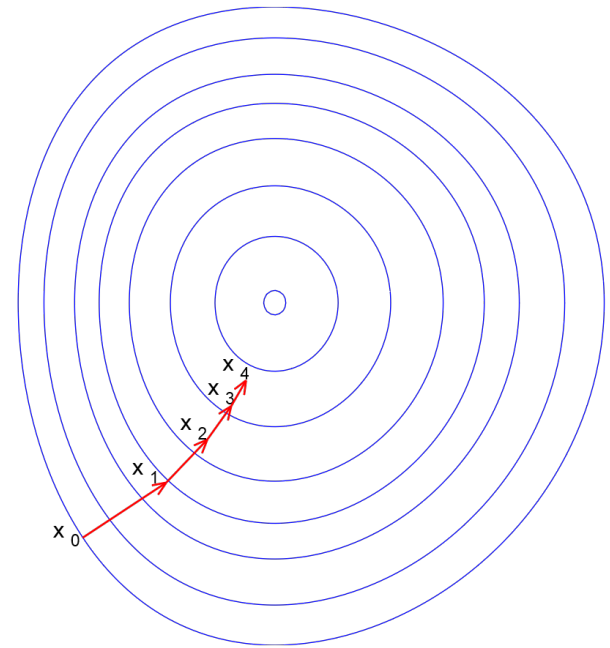
- $\hat{x}$ is the solution of the following equation

$$A^T Ax = A^T b$$

Numerical methods

- If no analytical solution?
 - Find a solution path such that

$$F(x_0) > F(x_1) > \dots > F(x_k) > \dots$$



Numerical methods

$\mathbf{x} \leftarrow \mathbf{x}_0$ % Initialization

while not converge

$\mathbf{h} \leftarrow \text{descending_direction}(\mathbf{x})$ % *determine the direction*

$\alpha \leftarrow \text{descending_step}(\mathbf{x}, \mathbf{h})$ % *determine the step*

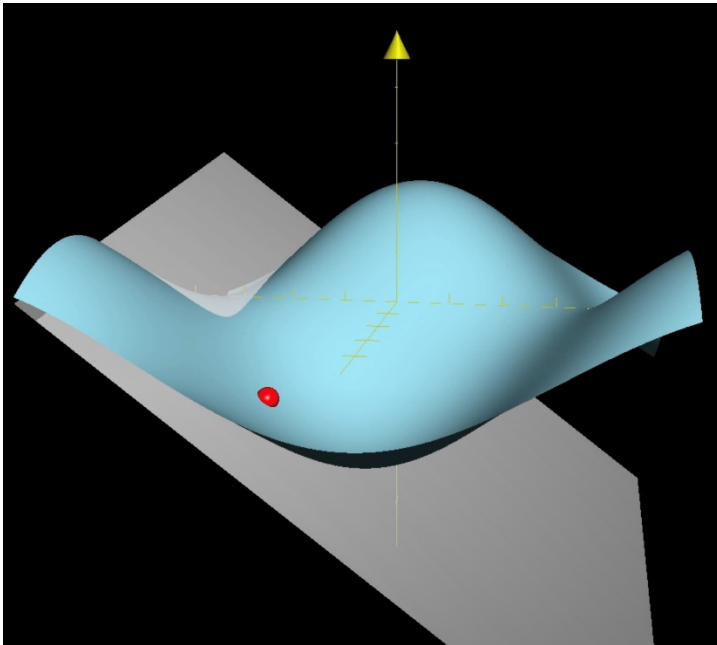
$\mathbf{x} \leftarrow \mathbf{x} + \alpha \mathbf{h}$ % *update the parameters*

How to find the best $\mathbf{h}$ and α ?

Recap: Taylor expansion

- First-order approximation

$$F(x_k + \Delta x) \approx F(x_k) + J_F \Delta x$$



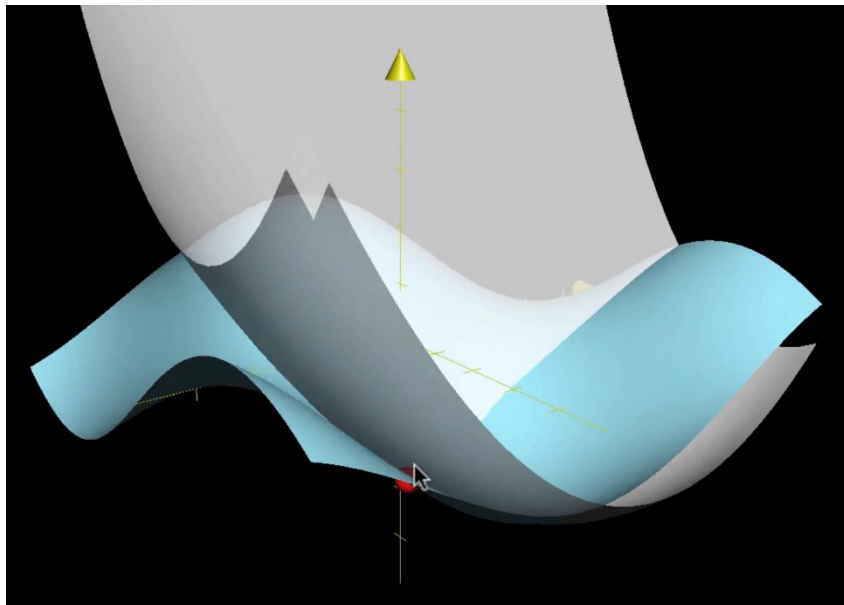
$$\mathbf{J} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}}{\partial x_n} \end{bmatrix}$$

Jacobian matrix
(first-order derivative)

Recap: Taylor expansion

- Second-order approximation

$$F(x_k + \Delta x) \approx F(x_k) + J_F \Delta x + \frac{1}{2} \Delta x^T H_F \Delta x$$



$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}.$$

Hessian matrix
(second-order derivative)

Gradient descent (GD)

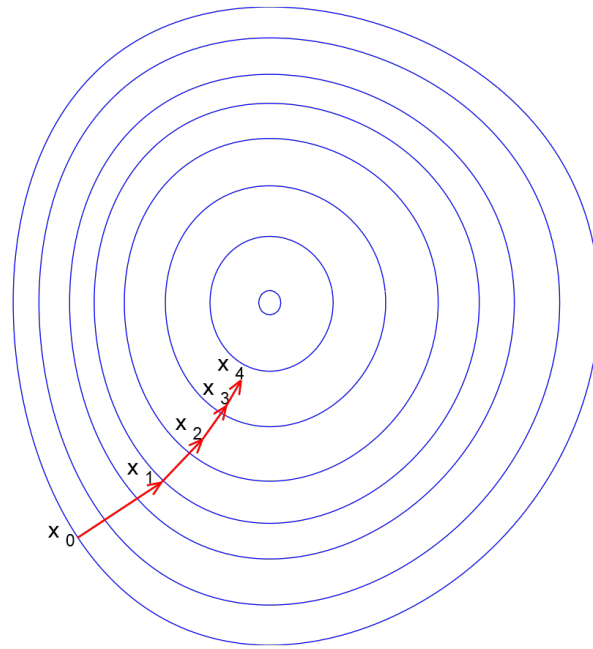
- Determine the direction by first-order approximation
 - Taylor expansion at x_0

$$F(x_0 + \Delta x) \approx F(x_0) + J_F \Delta x$$

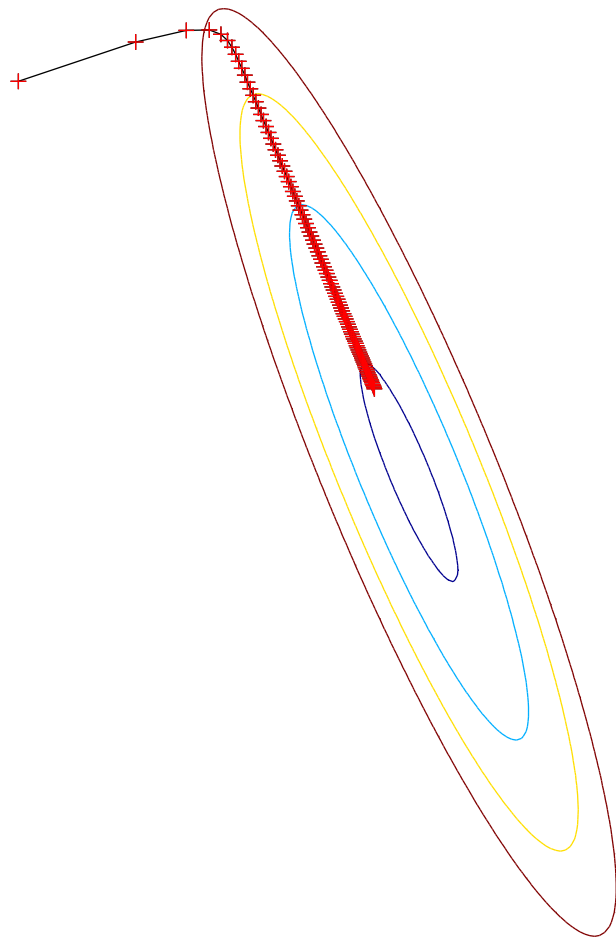
- When $J_F \Delta x < 0$, the object function will descend

Steepest descent method

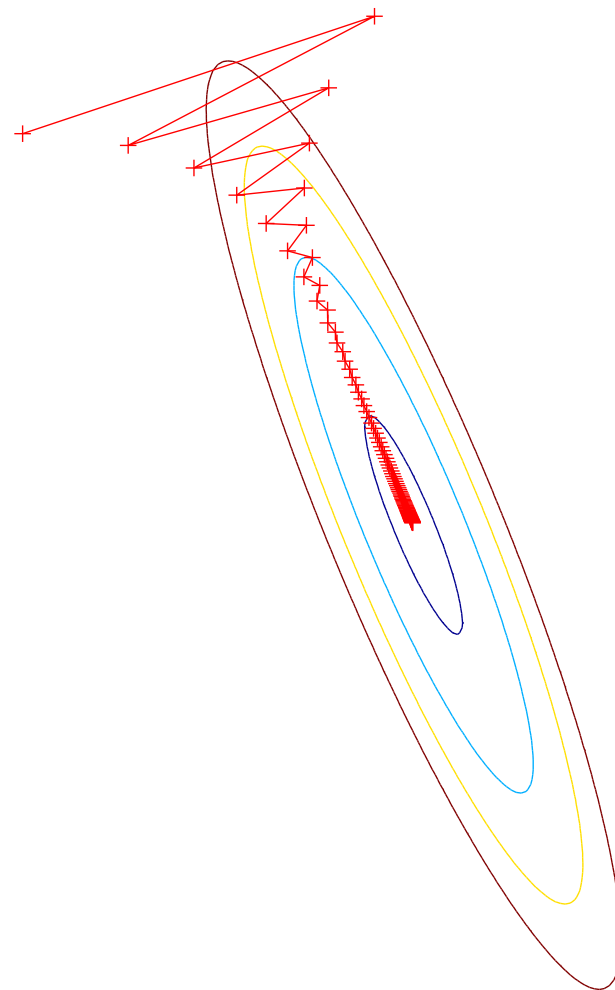
- When direction of Δx is same as $-J_F^T$, the objective function descend steepest



The impact of the step size



Too small



Too big

Determine the step size

α is small, the change of $\phi(\alpha)$ is small

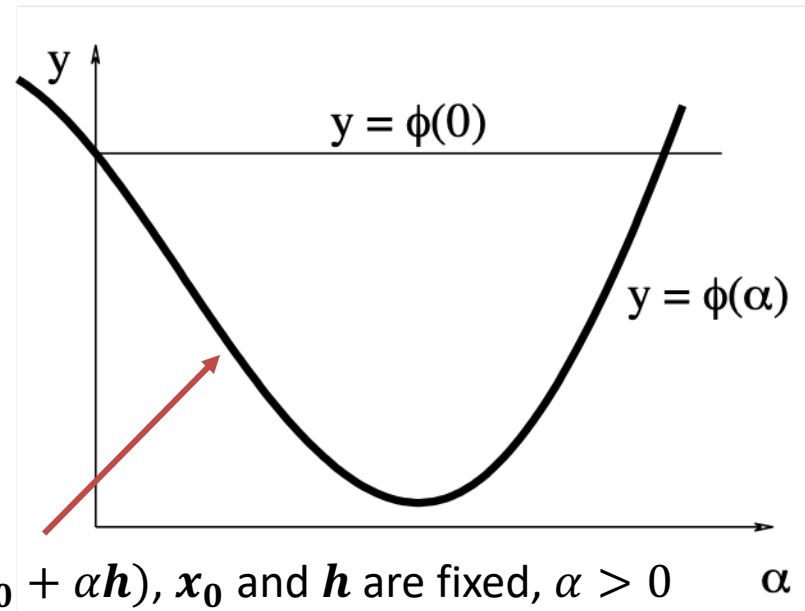
-> increase α

α is big, $\phi(\alpha) > \phi(0)$

-> decrease α

α is close to $\phi(\alpha)$ minimizer

-> α is acceptable

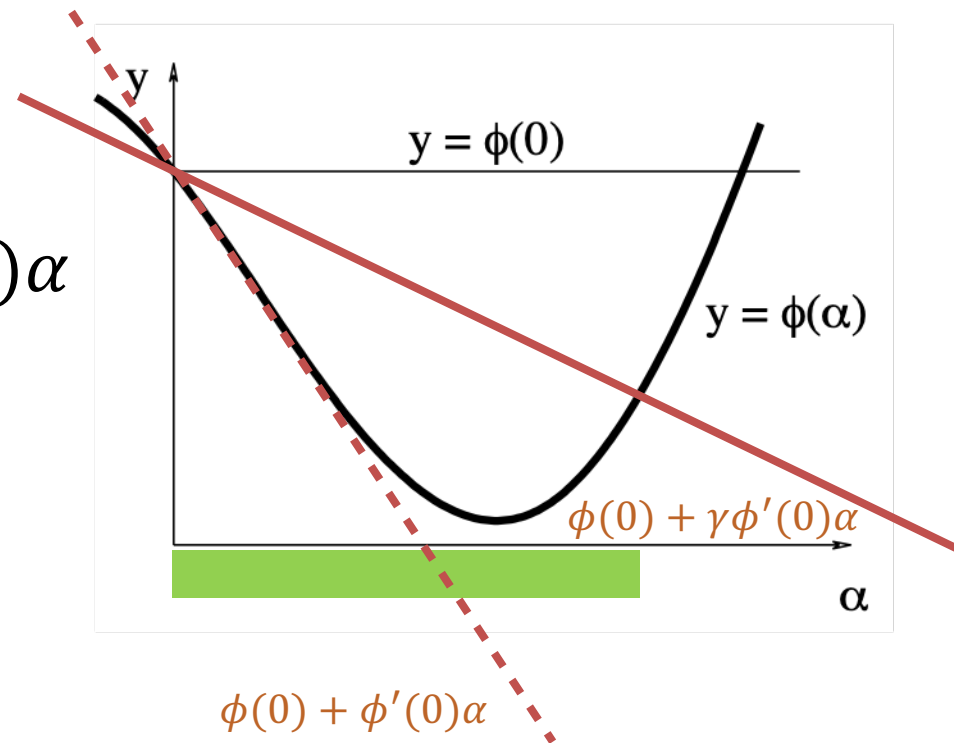


Determine the step size

- Exact line search
- Backtracking algorithm
 - Initialize α with a big value
 - Decrease α until

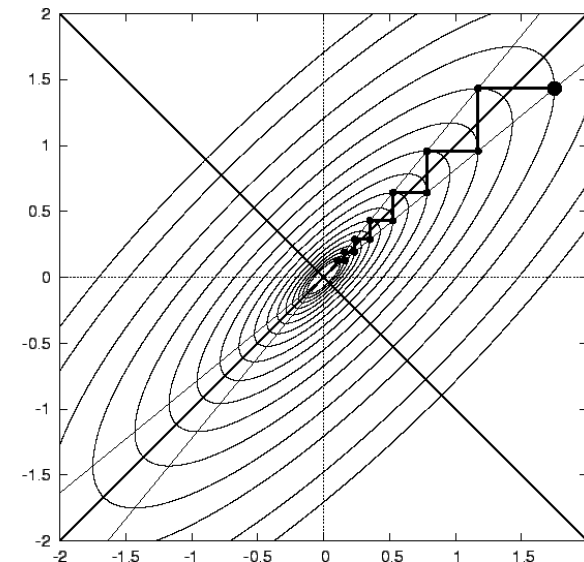
$$\phi(\alpha) \leq \phi(0) + \gamma \phi'(0)\alpha$$

- γ is a parameter, $0 < \gamma < 1$

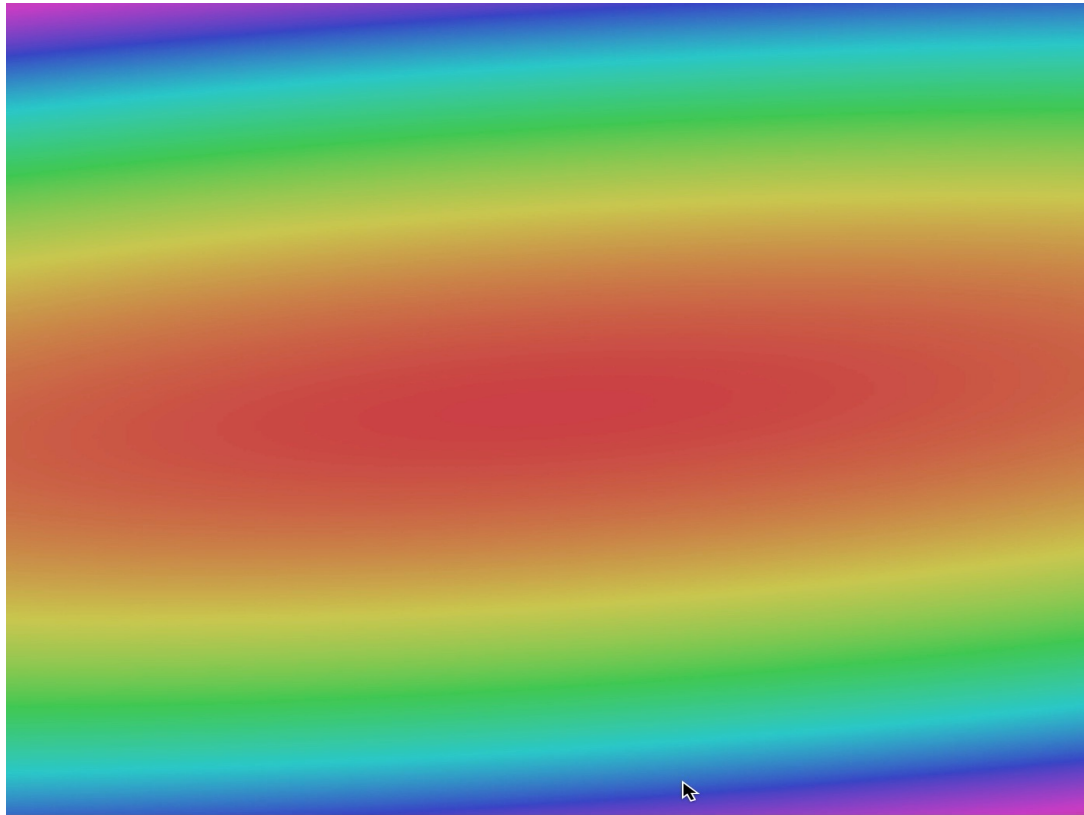


Steepest descent method

- Advantage
 - Easy to implement
 - Perform well when far from the minimum
- Disadvantage
 - Converge slowly when near the minimum
 - Waste a lot of computation
- Why converge slowly?
 - Only use first-order derivative
 - Does not use curvature



Steepest descent method



Newton method

- Do second-order expansion

$$F(x_k + \Delta x) \approx F(x_k) + J_F \Delta x + \frac{1}{2} \Delta x^T H_F \Delta x$$

- Find Δx to minimize $F(x_k + \Delta x)$

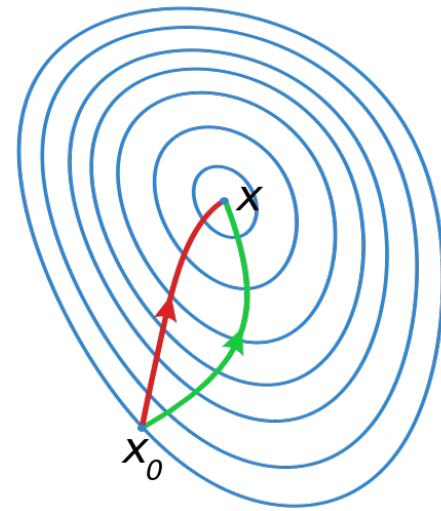
$$H_F \Delta x + J_F^T = 0$$

- The optimal direction (Newton step)

$$\Delta x = -H_F^{-1} J_F^T$$

Newton method

- Advantage
 - Fast convergence near the minimum
- Disadvantage
 - Hessian requires a lot of computation
- Can we approximate Hessian?



Green: GD
Red: Newton

Gauss-Newton method

- Useful for solving nonlinear least squares

$$\begin{aligned}\hat{x} &= \operatorname{argmin}_x F(x) \\ &= \operatorname{argmin}_x \|R(x)\|_2^2\end{aligned}$$

$$R(x) = \begin{bmatrix} b_1 - f_x(a_1) \\ b_2 - f_x(a_2) \\ \vdots \\ b_n - f_x(a_n) \end{bmatrix} \text{ is called the residual vector}$$

Gauss-Newton method

- Useful for solving nonlinear least squares

$$\hat{x} = \underset{x}{\operatorname{argmin}} \|R(x)\|_2^2$$

- Instead of expanding $F(x)$, we expand $R(x)$

$$\begin{aligned}\|R(x_k + \Delta x)\|_2^2 &\approx \|R(x_k) + J_R \Delta x\|_2^2 && J_R \text{ is the Jacobian of } R(x) \\ &= \|R(x_k)\|_2^2 + 2R(x_k)^T J_R \Delta x + \Delta x^T J_R^T J_R \Delta x\end{aligned}$$

- Optimal Δx satisfies

$$J_R^T J_R \Delta x + J_R^T R(x_k) = 0$$

Gauss-Newton method

- Optimal direction

$$\Delta x = -(J_R^T J_R)^{-1} J_R^T R(x_k)$$

- Compared with Newton
 - Newton step: $\Delta x = -H_F^{-1} J_F^T = -H_F^{-1} J_R^T R(x_k)$
 - Gauss-Newton use $J_R^T J_R$ to approximate Hessian H_F

Gauss-Newton method

- Advantage
 - No need to compute Hessian
 - Fast to converge
- Disadvantage
 - If $J_R^T J_R$ is singular, the algorithm becomes unstable

Levenberg-Marquardt

- LM employ regularization to overcome this

$$\Delta x = -(J_R^T J_R + \lambda I)^{-1} J_R^T R(x_k)$$

- For all $\lambda > 0$, $J_R^T J_R + \lambda I$ must be positive-definite

Levenberg-Marquardt

$$\Delta x = -(J_R^T J_R + \lambda I)^{-1} J_R^T R(x_k)$$

- The effect of λ
 - $\lambda \rightarrow \infty$: Gradient descent, and stepsize is small
 - $\lambda \rightarrow 0$: Gauss-Newton step

Levenberg-Marquardt

- Advantage
 - Start quickly ($\lambda \uparrow$)
 - Converge quickly ($\lambda \downarrow$)
 - Do not degenerate ($J_R^T J_R + \lambda I$ is always positive-definite)
 - LM = Gradient descent + Gauss-Newton

Constrained optimization

- Objective + constraints
- Different methods for different types of constraints

Constrained optimization

In mathematical optimization, **constrained optimization** (in some contexts called **constraint optimization**) is the process of optimizing an objective function with respect to some variables in the presence of constraints on those variables. The objective function is either a cost function or energy function, which is to be minimized, or a reward function or utility function, which is to be maximized. Constraints can be either **hard constraints**, which set conditions for the variables that are required to be satisfied, or **soft constraints**, which have some variable values that are penalized in the objective function if, and based on the extent that, the conditions on the variables are not satisfied.

Contents

Relation to constraint-satisfaction problems

General form

Solution methods

Equality constraints

Substitution method

Lagrange multiplier

Inequality constraints

Linear programming

Nonlinear programming

Quadratic programming

KKT conditions

Branch and bound

First-choice bounding functions

Russian doll search

Bucket elimination

See also

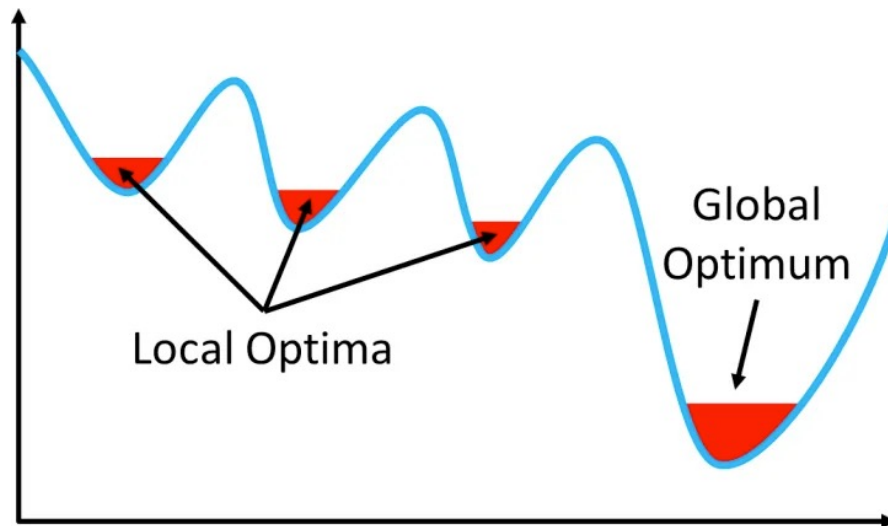
References

Further reading

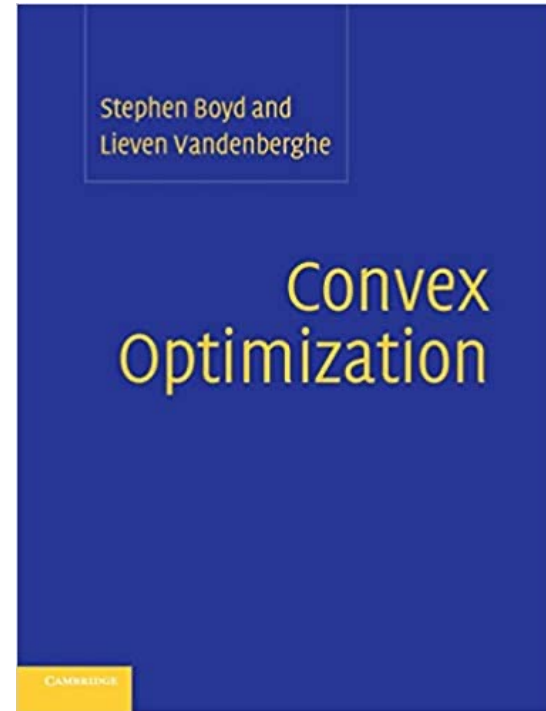
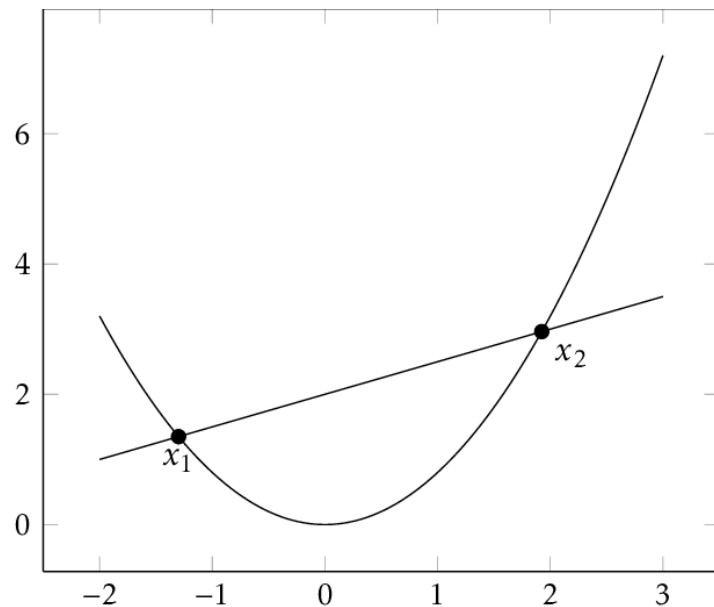
https://en.wikipedia.org/wiki/Constrained_optimization

Local minimum and global minimum

- GD can only find the local minimum
- For some functions, local minimum is global minimum



Convex optimization



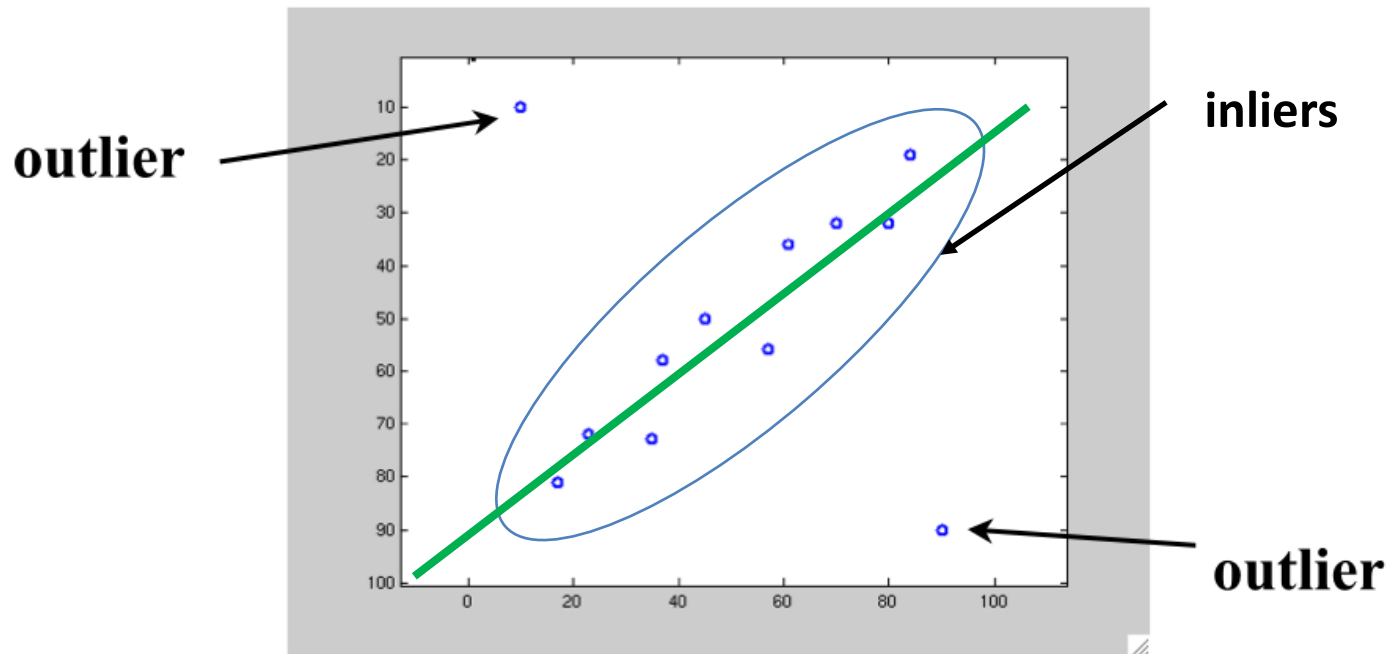
<https://web.stanford.edu/class/ee364a/>

Today

- Optimization
- Numerical methods
- **Robust estimation**
- Graphcut

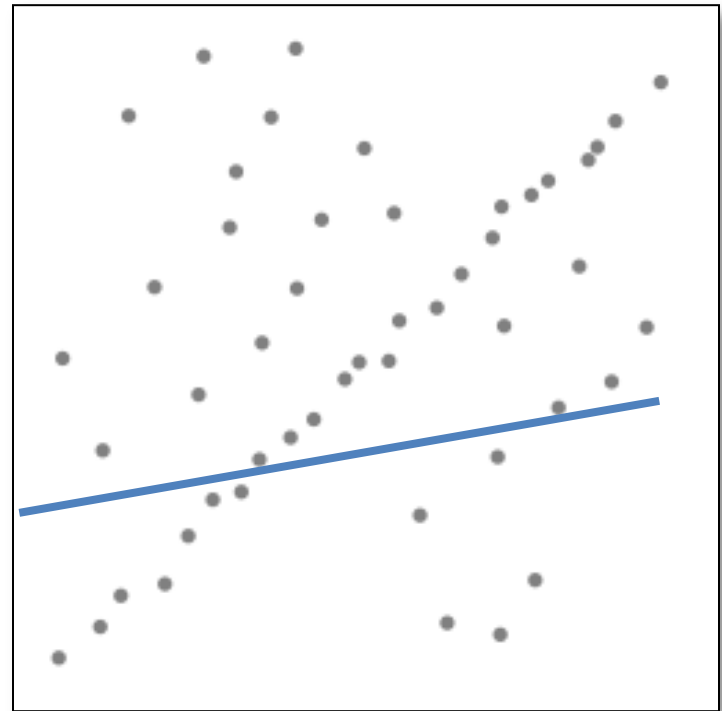
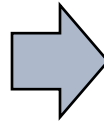
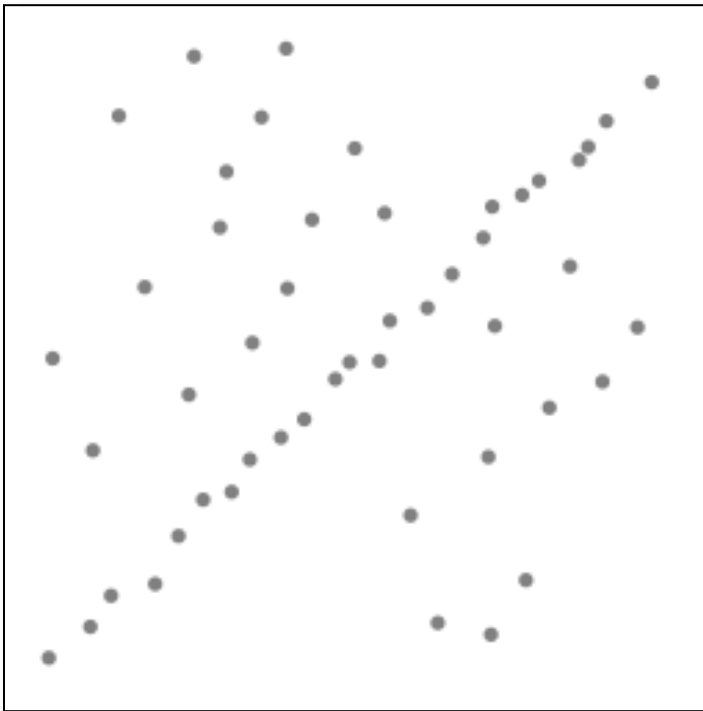
Outliers

- Inlier: obeys the model assumption
- Outlier: differs significantly from the assumption



Outliers

- Make MSE fail

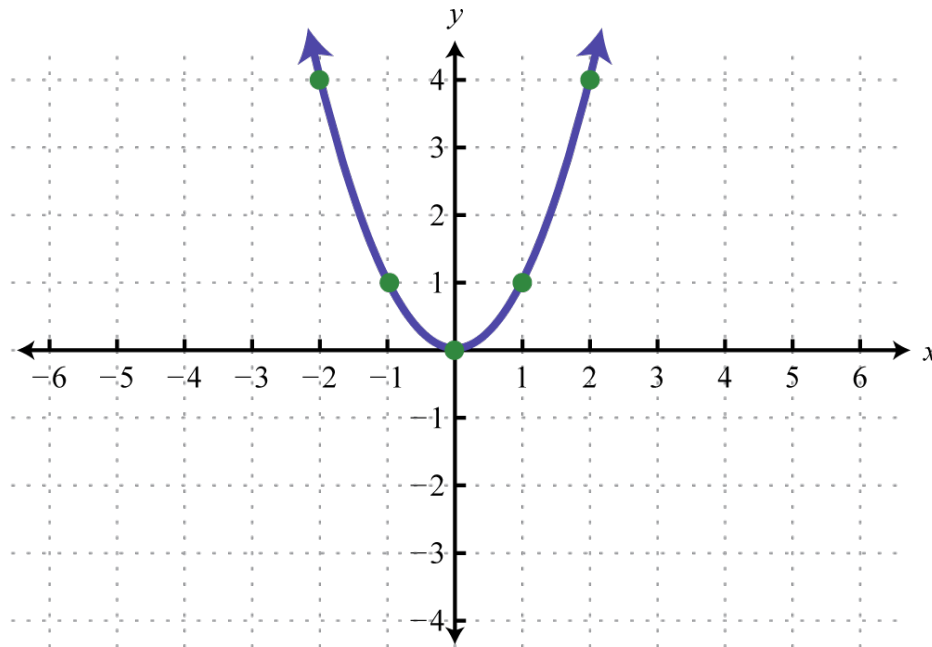


Outliers

- Why make MSE fail?
 - MSE is proportional to residual square
 - Affected a lot by outliers

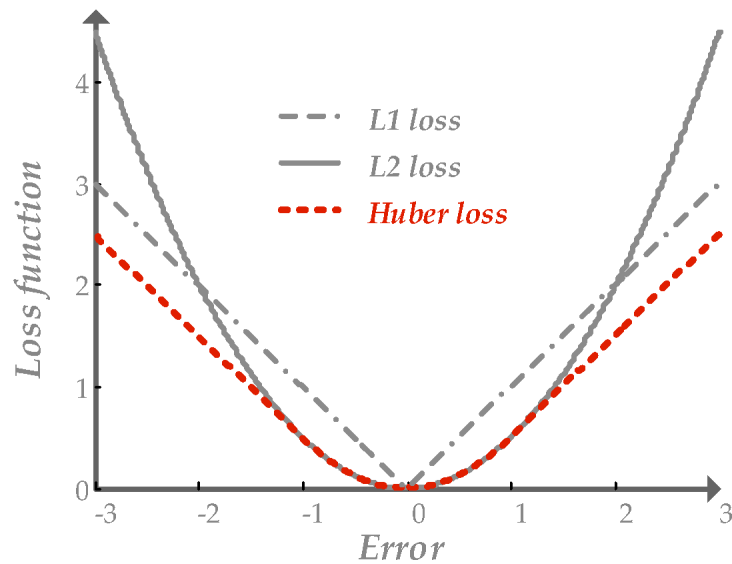
$$f(x) = x^2$$

x	$f(x)$
-2	4
-1	1
0	0
1	1
2	4



Robust estimation

- How to reduce the effect?
 - Use other loss functions to replace MSE
 - Like L1, Huber
 - They are called robust functions



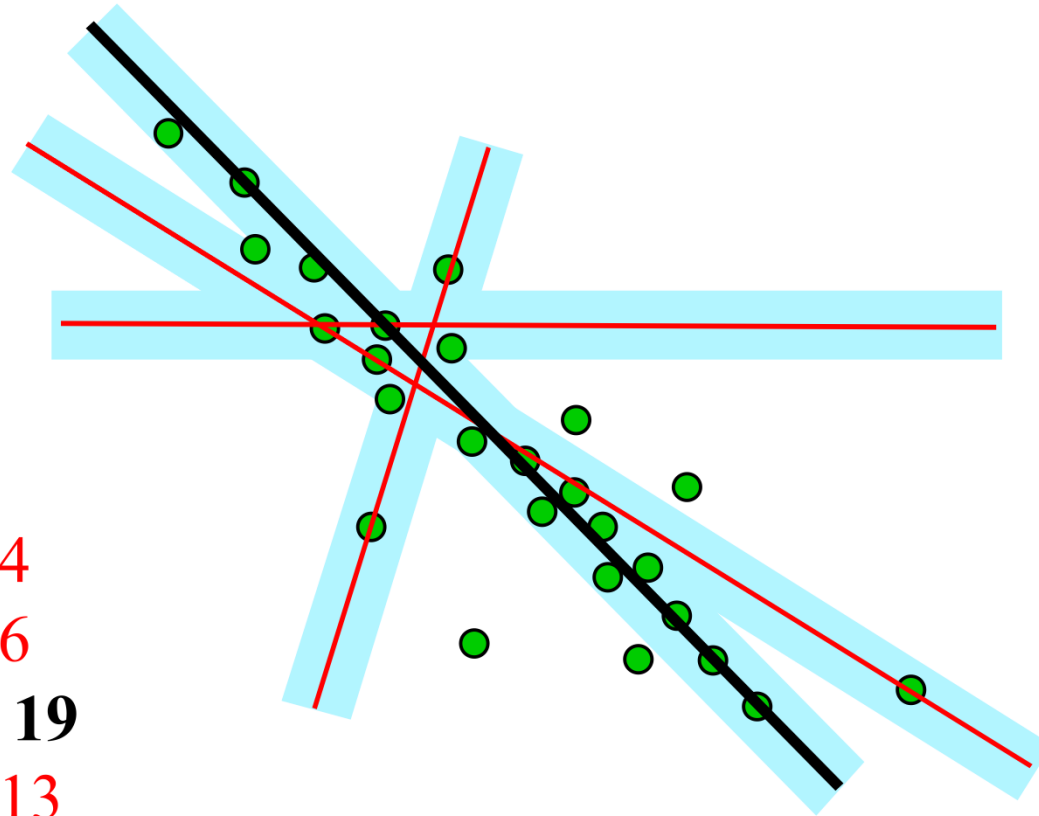
RANSAC

RANSAC: Random Sample Consensus

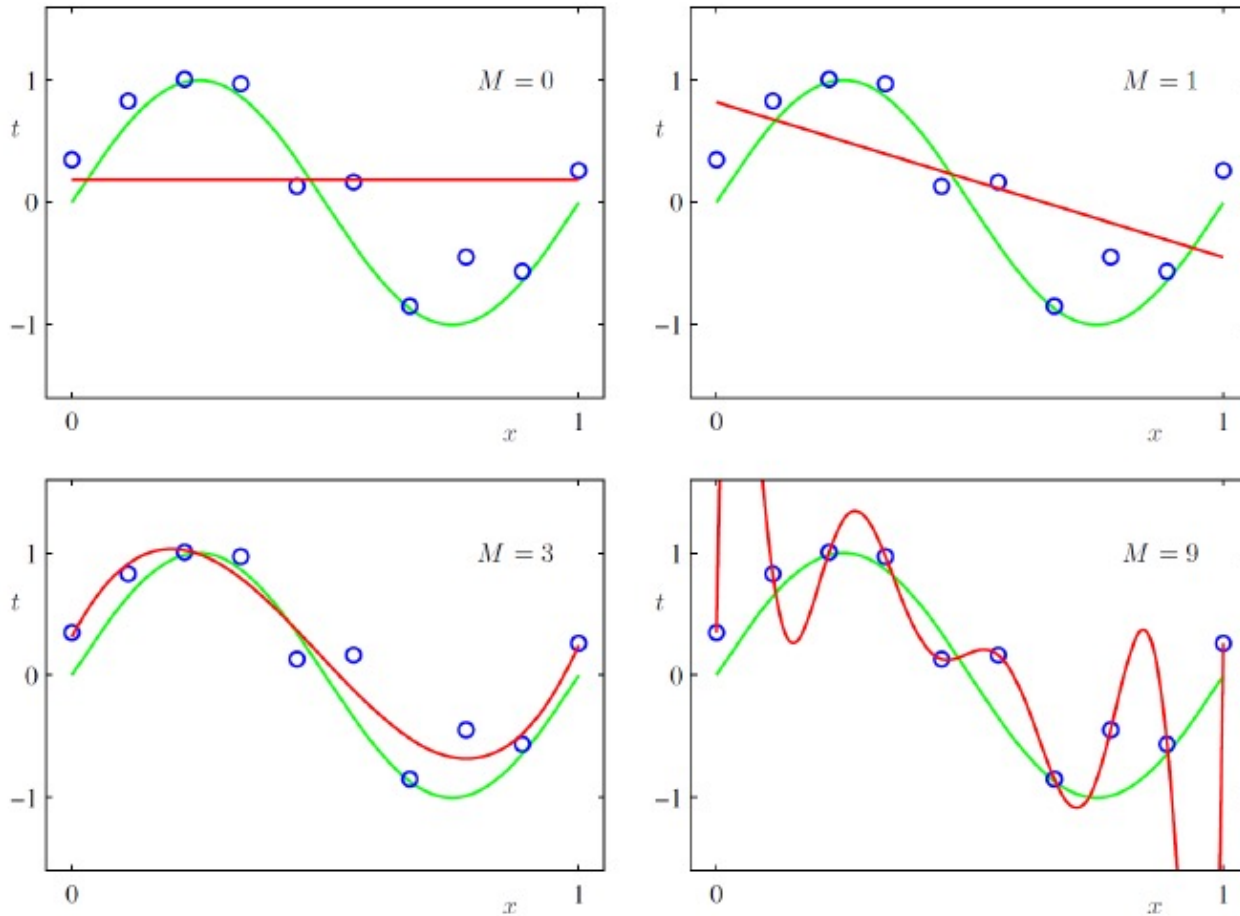
- The most powerful method to handle outliers
- Key ideas
 - The distribution of inliers is similar
while outliers differ a lot
 - Use data point pairs to vote

Ransac Procedure

Count = 4
Count = 6
Count = 19
Count = 13



Overfitting and underfitting



ill-posed problem

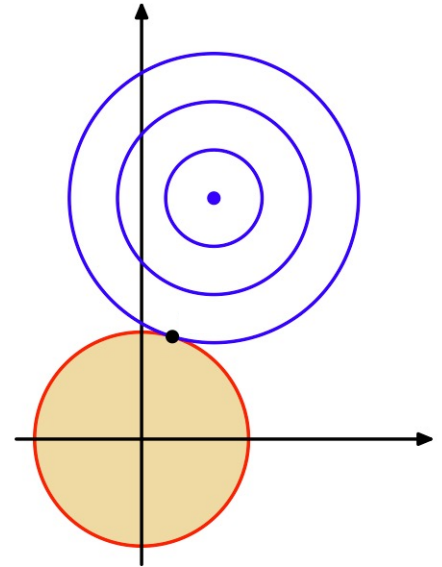
- The solution is not unique
 - Example:
 - $\min_x ||Ax - b||^2$ when #equations is less than #variables
- How to make the solution unique?
 - use prior knowledge to add more constraints

L2 regularization

- L2 norm: $\|x\|_2 = \sqrt{\sum_i x_i^2}$
- L2 regularization:

$$\begin{aligned} \min_x & \|Ax - b\|_2^2 \\ \text{s.t. } & \|x\|_2 \leq 1 \end{aligned}$$

- Make $x \rightarrow 0$
- Suppress redundant variables

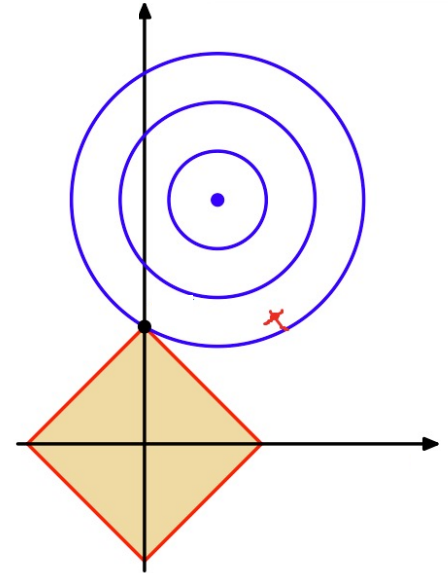


L1 regularization

- L1 norm: $\|x\|_1 = \sum_i |x_i|$
- L1 regularization:

$$\begin{aligned} \min_x & \|Ax - b\|_2^2 \\ \text{s.t. } & \|x\|_1 \leq 1 \end{aligned}$$

– Make x sparse



Today

- Optimization
- Numerical methods
- Robust estimation
- **Graphcut**

Image labeling problems

- Assign a label to each image pixel
 - e.g., foreground or background

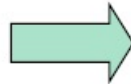


Image labeling problems

- Neighboring pixels tend to take the same label

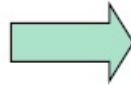


Image labeling problems

- How to model this prior?

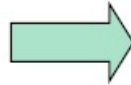


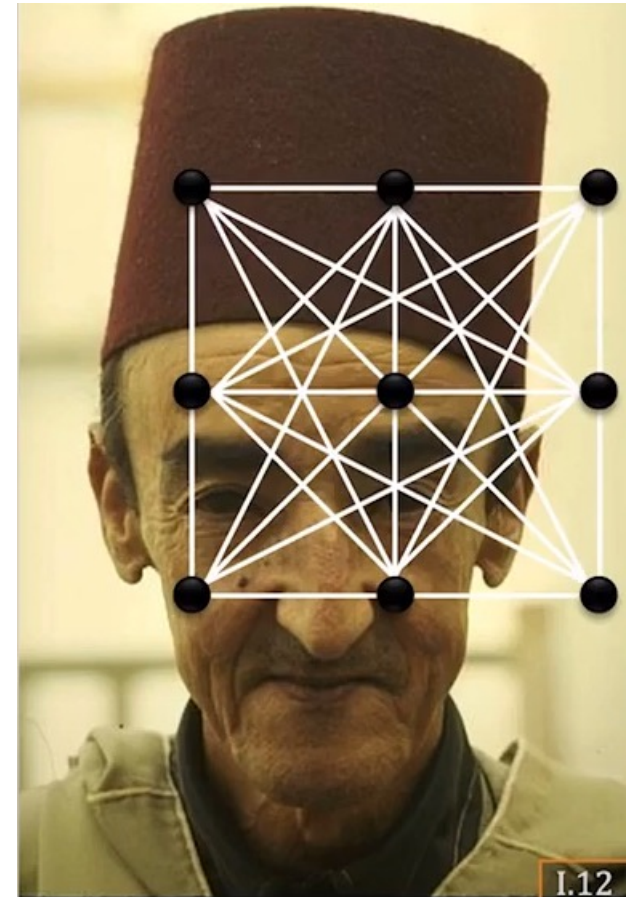
Image labeling problems

- Images as Graphs
 - A vertex for each pixel



Image labeling problems

- Images as Graphs
 - A vertex for each pixel
 - An edge between each pair
 - Each edge is weighted by the affinity or similarity between its two vertices



Measuring affinity

Let i and j be two pixels whose features are $\mathbf{f}_i$ and $\mathbf{f}_j$

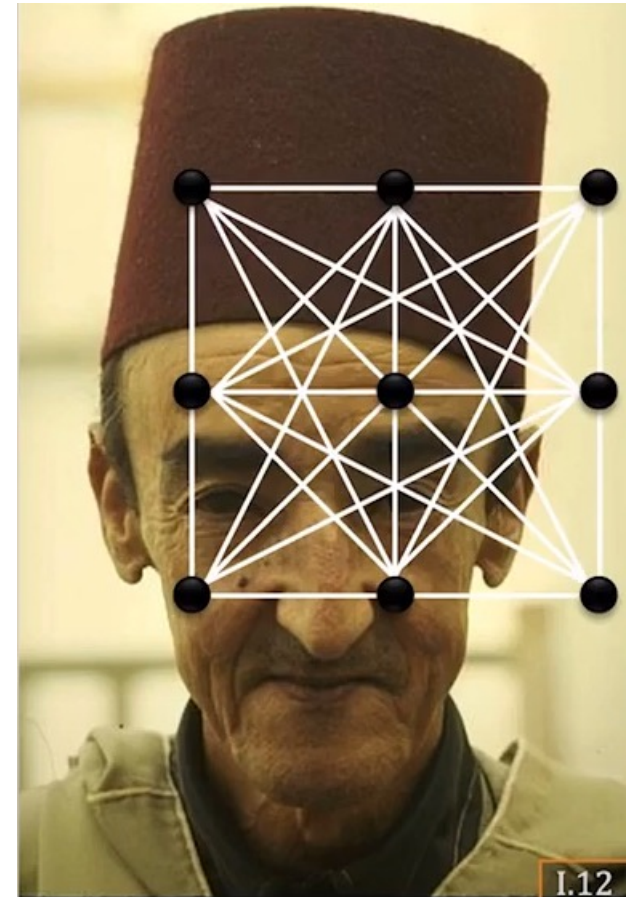
– Example: f can be color

- Pixel Dissimilarity:

$$S(\mathbf{f}_i, \mathbf{f}_j) = \sqrt{\left(\sum_k (f_{ik} - f_{jk})^2 \right)}$$

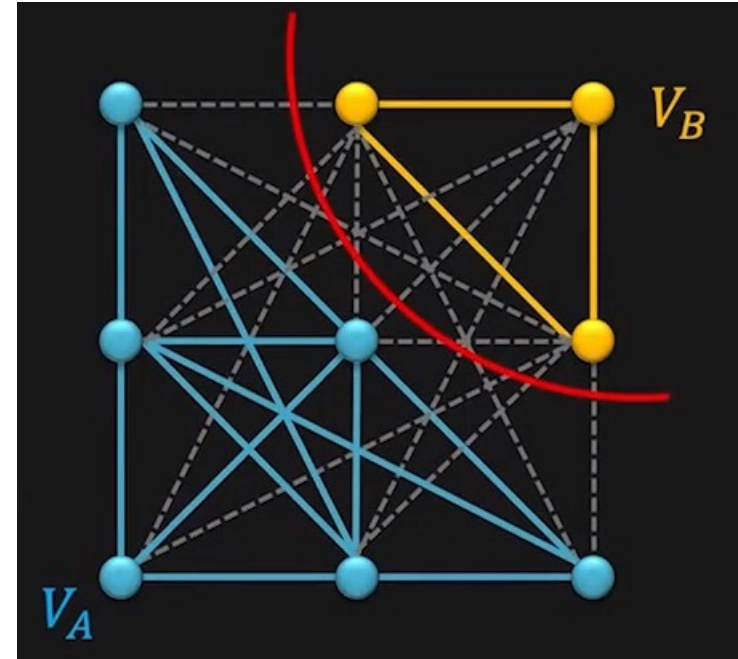
- Pixel Affinity:

$$w(i, j) = A(\mathbf{f}_i, \mathbf{f}_j) = e^{\left\{ \frac{-1}{2\sigma^2} S(\mathbf{f}_i, \mathbf{f}_j) \right\}}$$



Graph cut

- Cut $C = (V_A, V_B)$ is a partition of vertices V of a graph G into two disjoint subsets V_A and V_B

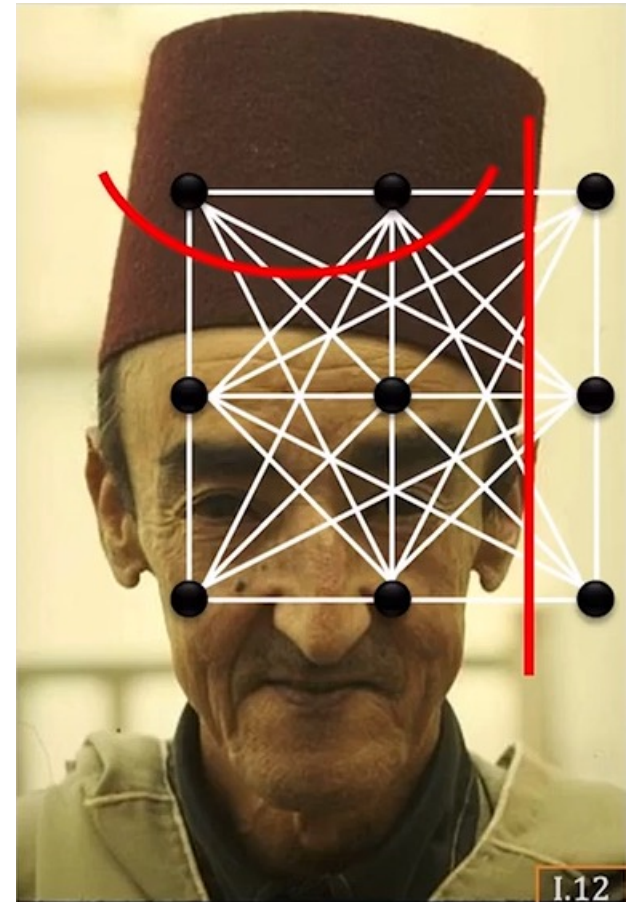


- Cost of Cut: Sum of weights of cut-set edges

$$\text{cut}(V_A, V_B) = \sum_{u \in V_A, v \in V_B} w(u, v)$$

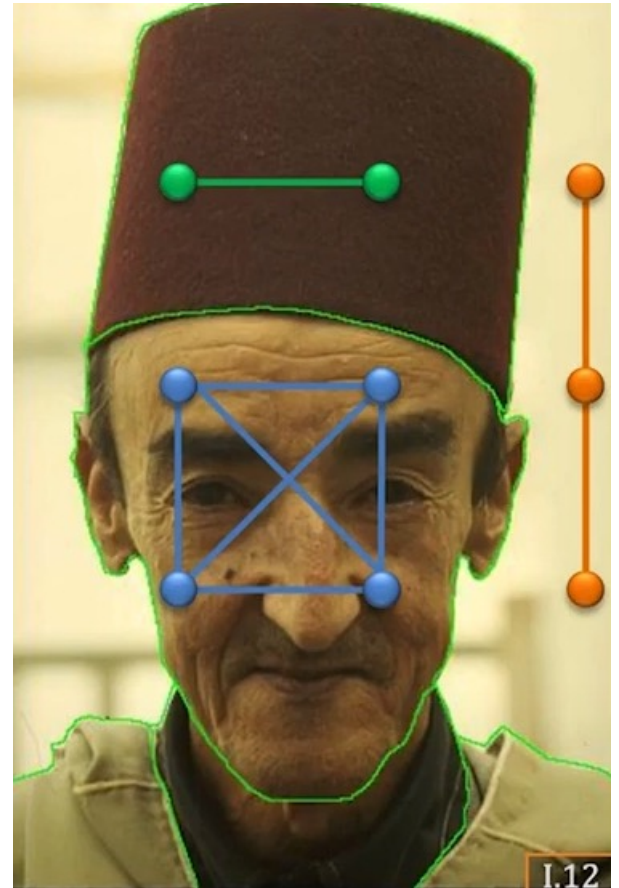
Graph cut

- Segmentation as graph cut
 - Vertices within a subgraph have high affinity
 - Vertices from two different subgraphs have low affinity
- Minimize the cost of cut



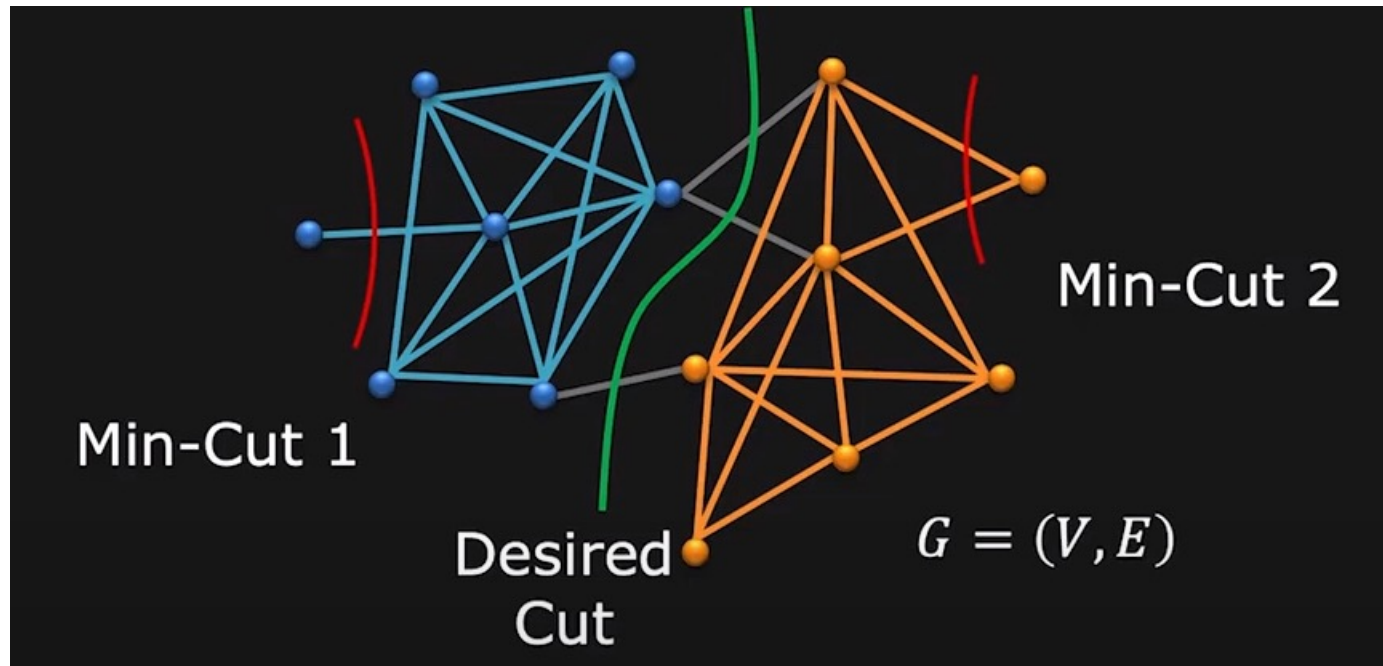
Graph cut

- Each subgraph is an image segment



Problem with min-cut

- Bias to cut small, isolated segments



Normalized cut

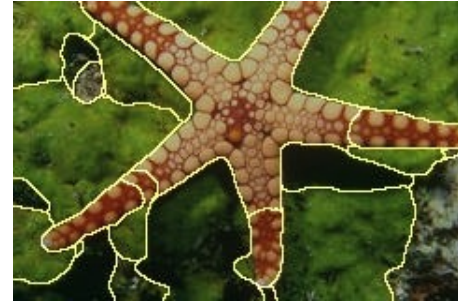
- Normalizing for size of segments:

$$\text{NCut}(V_A, V_B) = \frac{\text{cut}(V_A, V_B)}{\text{assoc}(V_A, V)} + \frac{\text{cut}(V_A, V_B)}{\text{assoc}(V_B, V)}$$

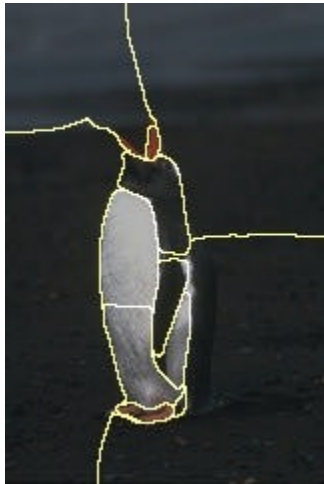
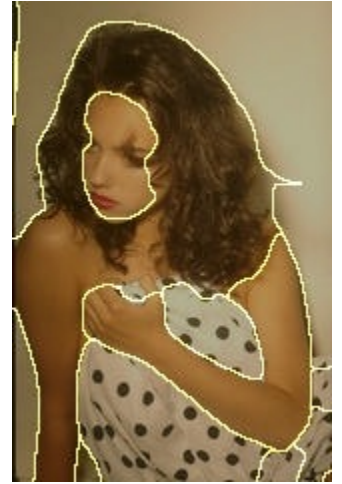
$$\text{assoc}(V_A, V) = \sum_{u \in V_A, v \in V} w(u, v)$$

- NP-Complete
- Approximate solution by eigenvalue decomposition

Examples



Examples



Questions?